Full Regression Test Report

1. Project Information

Project Name	Fort Point Properties
Owner	Zander N. Aligato
Date	May 8, 2026
Branch Name	refactor/vertical_slice_refactoring
Repository Link	https://github.com/zandruhux/IT342-Aligato-FortPointProperties
Scope	Backend (Spring Boot), Web (React/Vite), Mobile (Android/Kotlin)

2. Refactoring Summary

Fort Point Properties was refactored using Vertical Slice Architecture. Before the refactoring, the project used the traditional Layered (N-Tier) Architecture, which focused on the separation of concerns. After refactoring, the files were organized by feature or module which means that it may be able to run independently without having to affect other features.

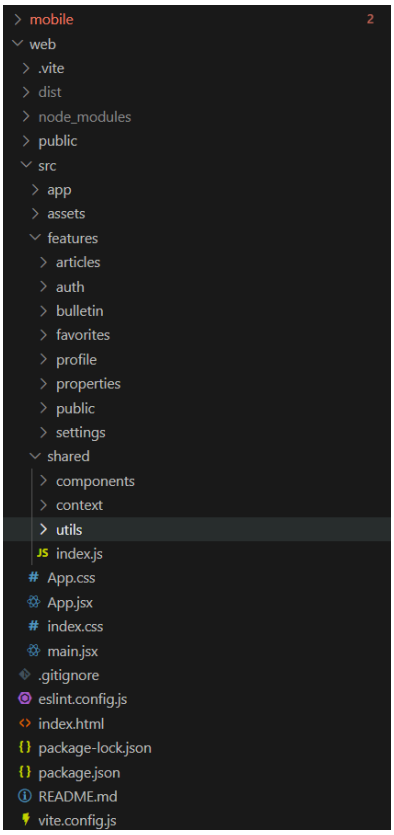
The refactoring was applied to the backend, web frontend, and mobile. All existing features were tested and verified after testing to see if the system still works as intended.

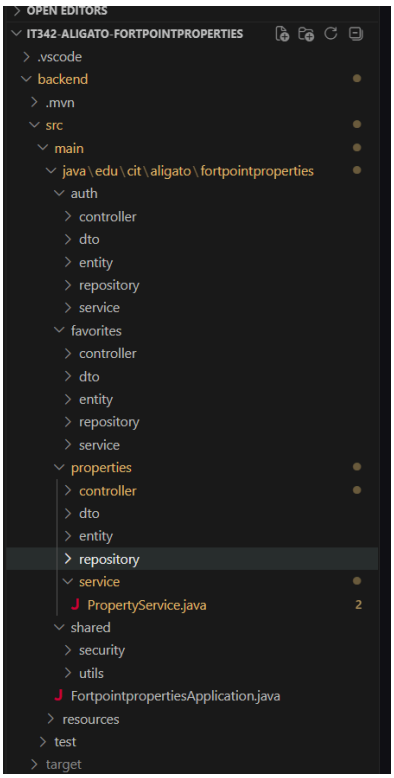
3. Updated Project Structure

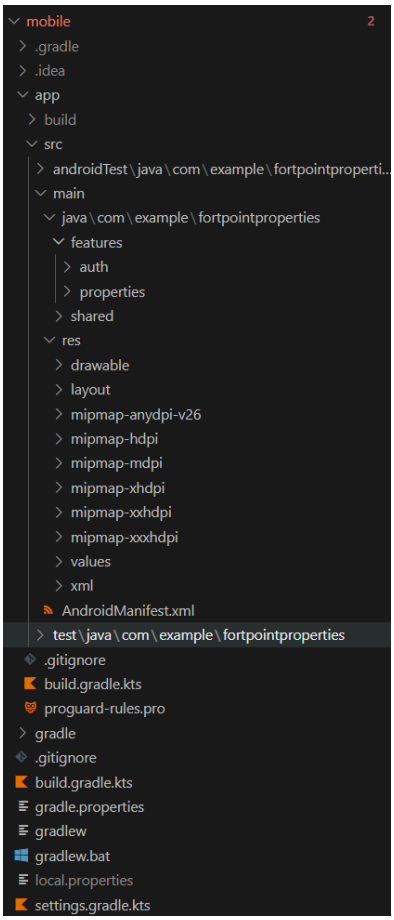
Figure 1:
Frontend Vertical Slice Structure

Figure 2:
Backend Vertical Slice Structure

Figure 3:
Mobile Vertical Slice Structure







Others folders serves as placeholders, no files implemented yet

4. Test Plan Documentation

4.1 Functional Requirements Coverage

The following list provides a detailed breakdown of functional requirements and their corresponding test case identifiers, organized by system module.

Authentication

- User can register with valid details (TC-AUTH-001, TC-AUTH-002)
- User cannot register with duplicate email or invalid password (TC-AUTH-003, TC-AUTH-004)
- User can login and receive valid session tokens (TC-AUTH-005, TC-AUTH-006)
- Authenticated user can fetch own profile (TC-AUTH-007)
- Unauthorized/invalid token access is blocked (TC-SEC-001, TC-SEC-002)

Property Management

- Public user can view public property list (TC-PROP-001)

- Registered user can view extended property details (TC-PROP-002)
- Agent can view agent property list/details and search (TC-PROP-003, TC-PROP-004)
- Admin can create, update, delete properties (TC-PROP-005, TC-PROP-006, TC-PROP-007)
- Admin can create, update, delete property units (TC-PROP-008, TC-PROP-009)

Search

- Search/filter by name/location/developer/price behaves correctly (TC-PROP-010, TC-PROP-011)

Favorites

- Registered user can add/remove favorites (TC-FAV-001, TC-FAV-002)
- Registered user can view favorites list and count (TC-FAV-003)

Protected Routes

- Role-based routing and sidebar access is enforced (TC-ROLE-001, TC-ROLE-002)

Mobile Authentication

- Mobile app launches auth flow and supports profile/logout (TC-MOB-001, TC-MOB-002)

4.2 Test Cases and Test Scripts

Test Cases and Test Scripts Details

Test Case ID	Module	Pre-conditions	Test Steps	Expected Result	Automated / Manual
TC-AUTH-001	Authentication	User email does not exist.	1. Open /register. 2. Fill first name, last name, email, strong password, confirm password. 3. Submit form.	1. API returns success. 2. User session is initialized. 3. UI shows success and redirects according to role.	Automated
TC-AUTH-002	Authentication	User email does not exist.	1. Open mobile Register screen. 2. Fill all required fields. 3. Tap Register.	1. Registration succeeds. 2. Tokens are saved. 3. User can proceed to login/profile.	Manual
TC-AUTH-003	Authentication	Existing account with target email.	1. Submit register request using existing email.	1. Backend returns conflict/error. 2. UI displays duplicate email message.	Automated

Test Case ID	Module	Pre-conditions	Test Steps	Expected Result	Automated / Manual
TC-AUTH-004	Authentication	None.	1. Attempt registration with password < minimum policy.	1. Validation fails (frontend and/or backend). 2. Clear error message shown.	Automated
TC-AUTH-005	Authentication	Existing user credentials.	1. Submit valid login credentials.	1. Access/refresh tokens returned. 2. Session state updated. 3. Role-based redirection is applied.	Automated
TC-AUTH-006	Authentication	None.	1. Submit invalid email/password.	1. Login rejected. 2. Error message shown.	Automated
TC-AUTH-007	Authentication	Valid access token.	1. Call profile endpoint via web/mobile.	1. Profile fields are returned and rendered.	Automated
TC-SEC-001	Security and Access Control	No auth token.	1. Call /api/v1/auth/profile or /user/favorites.	1. Request is rejected (401/403).	Manual
TC-SEC-002	Security and Access Control	Logged in as USER or AGENT where ADMIN required.	1. Call /admin/properties.	1. Request is rejected by security config.	Manual
TC-PROP-001	Properties	Public user.	1. Open public properties page.	1. Public property list loads.	Manual
TC-PROP-002	Properties	Logged in as registered user.	1. Open property detail modal.	1. Allowed detail fields are visible. 2. Restricted fields are not exposed.	Manual
TC-PROP-003	Properties	Logged in as AGENT.	1. Open agent properties. 2. Open details for one property.	1. Agent list and advanced details load.	Manual
TC-PROP-004	Properties	Logged in as AGENT.	1. Search with developer filter.	1. Matching properties are returned.	Manual

Test Case ID	Module	Pre-conditions	Test Steps	Expected Result	Automated / Manual
TC-PROP-005	Properties	Logged in as ADMIN.	1. Open create property modal. 2. Fill required fields. 3. Submit.	1. Property created. 2. List refreshes with new item.	Automated
TC-PROP-006	Properties	Existing property, ADMIN role.	1. Open property details in edit mode. 2. Modify fields. 3. Save.	1. Property updates successfully. 2. Updated values appear in list/detail.	Automated
TC-PROP-007	Properties	Existing property, ADMIN role.	1. Trigger delete action. 2. Confirm prompt.	1. Property is removed. 2. List updates.	Automated
TC-PROP-008	Properties	Existing property, ADMIN role.	1. Open unit management section. 2. Add new unit or edit existing unit. 3. Save.	1. Unit persists and displays correctly.	Automated
TC-PROP-009	Properties	Existing property with units, ADMIN role.	1. Delete selected unit. 2. Confirm prompt.	1. Unit removed from detail view.	Automated
TC-PROP-010	Properties	Dataset with multiple locations.	1. Search using known location.	1. Returned properties match location criteria.	Automated
TC-PROP-011	Properties	Dataset with varied pricing.	1. Set min/max price filters.	1. Returned properties are within range logic.	Automated
TC-FAV-001	Favorites	Logged in as registered user.	1. Click favorite toggle on property card.	1. Favorite state turns on. 2. Backend records favorite.	Automated
TC-FAV-002	Favorites	Property already favorited.	1. Toggle favorite off.	1. Favorite removed from backend and UI.	Automated

Test Case ID	Module	Pre-conditions	Test Steps	Expected Result	Automated / Manual
TC-FAV-003	Favorites	Logged in as registered user with favorites.	1. Open favorites page.	1. Favorited properties and count are correct.	Automated
TC-ROLE-001	Role-Based Routing/UI	Logged in as ADMIN.	1. Navigate to admin pages.	1. Admin sidebar and admin routes accessible.	Manual
TC-ROLE-002	Role-Based Routing/UI	Logged in as AGENT.	1. Navigate to agent pages.	1. Agent sidebar and agent routes accessible. 2. Unauthorized admin actions not available.	Manual
TC-MOB-001	Mobile	Valid account.	1. Login via mobile.	1. Token saved. 2. Profile screen loads with user details.	Manual
TC-MOB-002	Mobile	Logged in mobile user.	1. Tap logout and confirm.	1. Token cleared. 2. User redirected to login screen.	Manual

The backend verification was conducted through an automated suite utilizing JUnit and Mockito. This comprehensive suite comprises AuthServiceTest, AuthControllerTest, FavoriteServiceTest, PropertyServiceTest, ApplicationContextTest, and FortpointPropertiesApplicationTests. These tests are designed to validate critical service-level business logic, the accuracy of controller response formatting, and the integrity of DTO mapping. Furthermore, they ensure the reliability of property and unit CRUD operations, the management of user favorites, and the successful initialization of the application startup context.

For the frontend, automated verification was facilitated via ESLint and Vite production build diagnostics. In contrast, the remaining UI flows for both web and mobile were subjected to manual testing, as specialized browser-based or mobile automation frameworks have not yet been integrated into these specific domains.

5. Regression Test Results

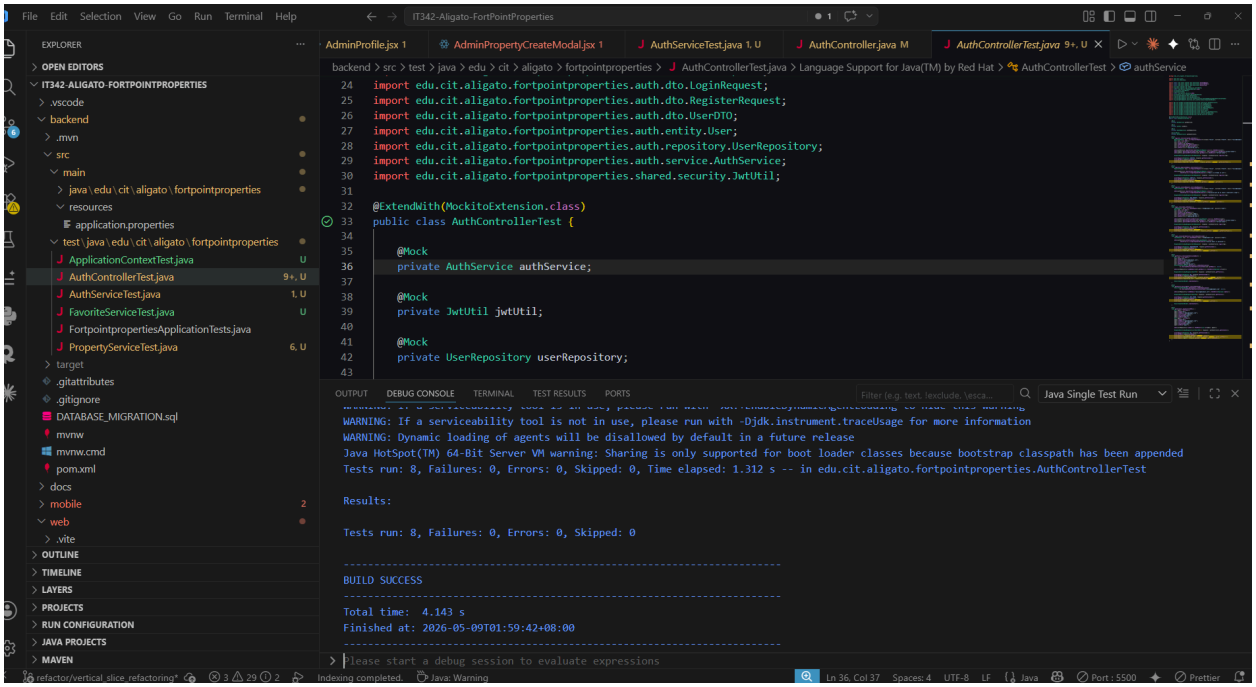
Test Case ID	Status	Tested By
TC-AUTH-001	PASS	Zander N. Aligato
TC-AUTH-002	PASS	Zander N. Aligato
TC-AUTH-003	PASS	Zander N. Aligato
TC-AUTH-004	PASS	Zander N. Aligato
TC-AUTH-005	PASS	Zander N. Aligato
TC-AUTH-006	PASS	Zander N. Aligato
TC-AUTH-007	PASS	Zander N. Aligato
TC-SEC-001	PASS	Zander N. Aligato
TC-SEC-002	PASS	Zander N. Aligato
TC-PROP-001	PASS	Zander N. Aligato
TC-PROP-002	PASS	Zander N. Aligato
TC-PROP-003	PASS	Zander N. Aligato
TC-PROP-004	PASS	Zander N. Aligato
TC-PROP-005	PASS	Zander N. Aligato
TC-PROP-006	PASS	Zander N. Aligato
TC-PROP-007	PASS	Zander N. Aligato
TC-PROP-008	PASS	Zander N. Aligato
TC-PROP-009	PASS	Zander N. Aligato
TC-PROP-010	PASS	Zander N. Aligato
TC-PROP-011	PASS	Zander N. Aligato
TC-FAV-001	PASS	Zander N. Aligato
TC-FAV-002	PASS	Zander N. Aligato
TC-FAV-003	PASS	Zander N. Aligato
TC-ROLE-001	PASS	Zander N. Aligato
TC-ROLE-002	PASS	Zander N. Aligato

Test Case ID	Status	Tested By
TC-MOB-001	PASS	Zander N. Aligato
TC-MOB-002	PASS	Zander N. Aligato

5.1 Automated Test Evidence

Backend Verification successful: Tests run: 46, Failures: 0, Errors: 0, Skipped: 0.

Figure 4: AuthController Test (8 Success, 0 Failures, 0 Errors, 0 Skipped)



Validates the operational logic of the authentication API controller. These tests confirm that the registration and login endpoints yield appropriate HTTP status codes, response payloads, session tokens, and error messaging. Furthermore, it verifies the retrieval of profile data within an authenticated user context, ensuring that internal user entities are accurately mapped to **UserDTO** objects for external responses.

Figure 5: AuthService Test (8 Success, 0 Failures, 0 Errors, 0 Skipped)

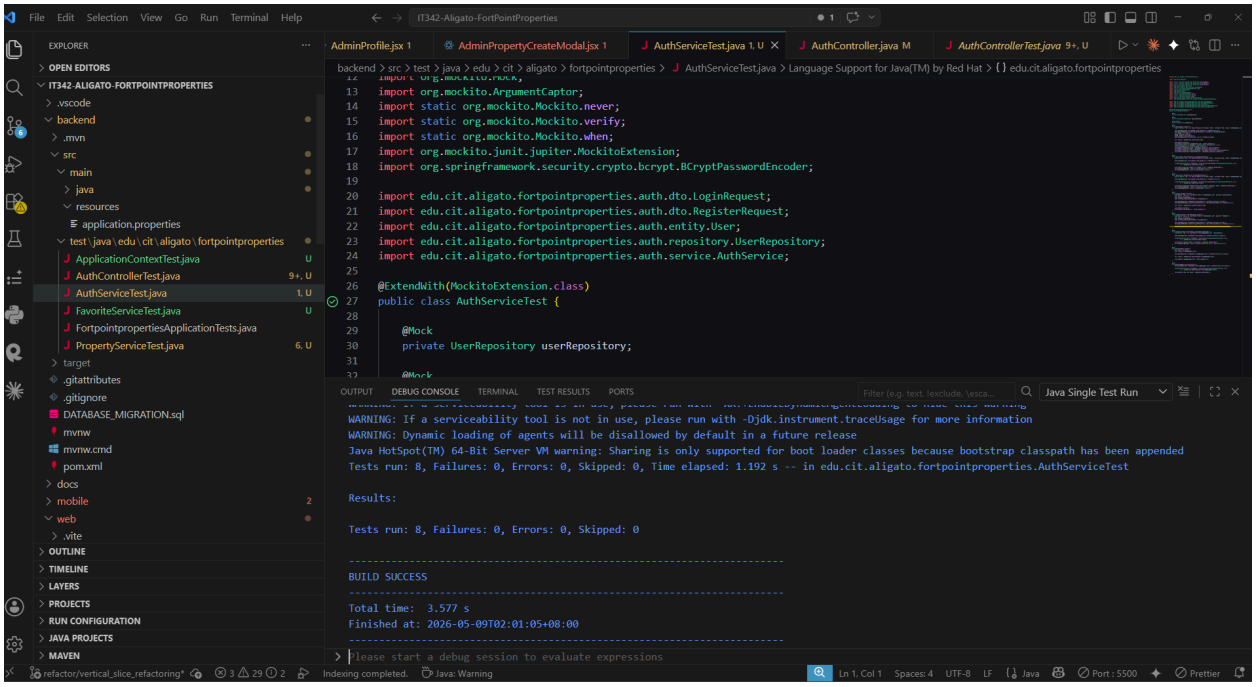


Figure 6: FavoriteService Test (9 Success, 0 Failures, 0 Errors, 0 Skipped)

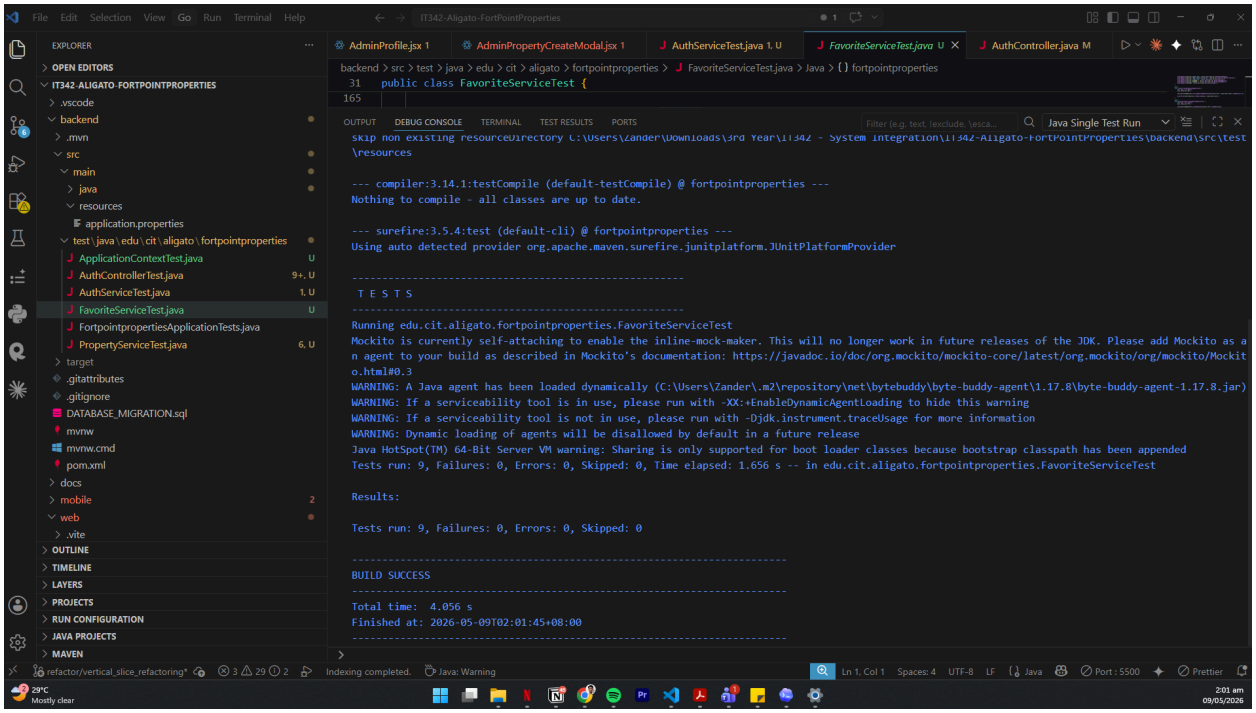


Figure 7: Property Service: 19 Success, 0 Failures, 0 Errors, 0 Skipped

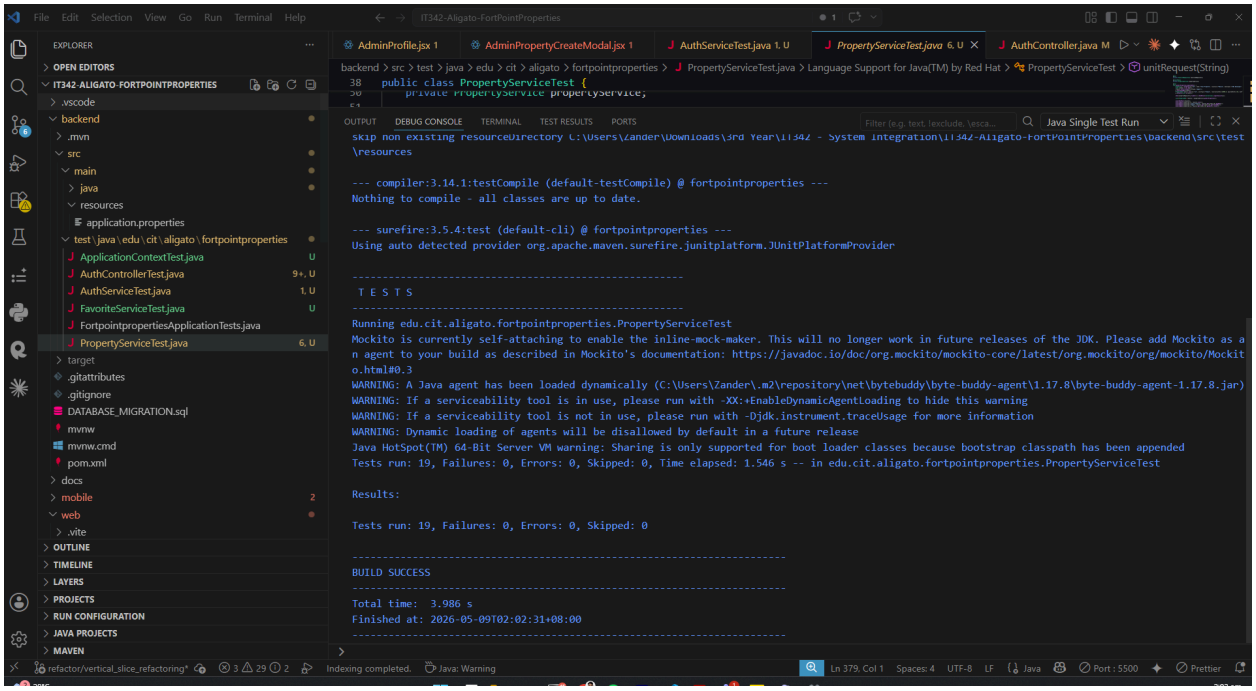


Figure 8: FortPointProperties Application Test (1 Success, 0 Failures, 0 Errors, 0 Skipped)

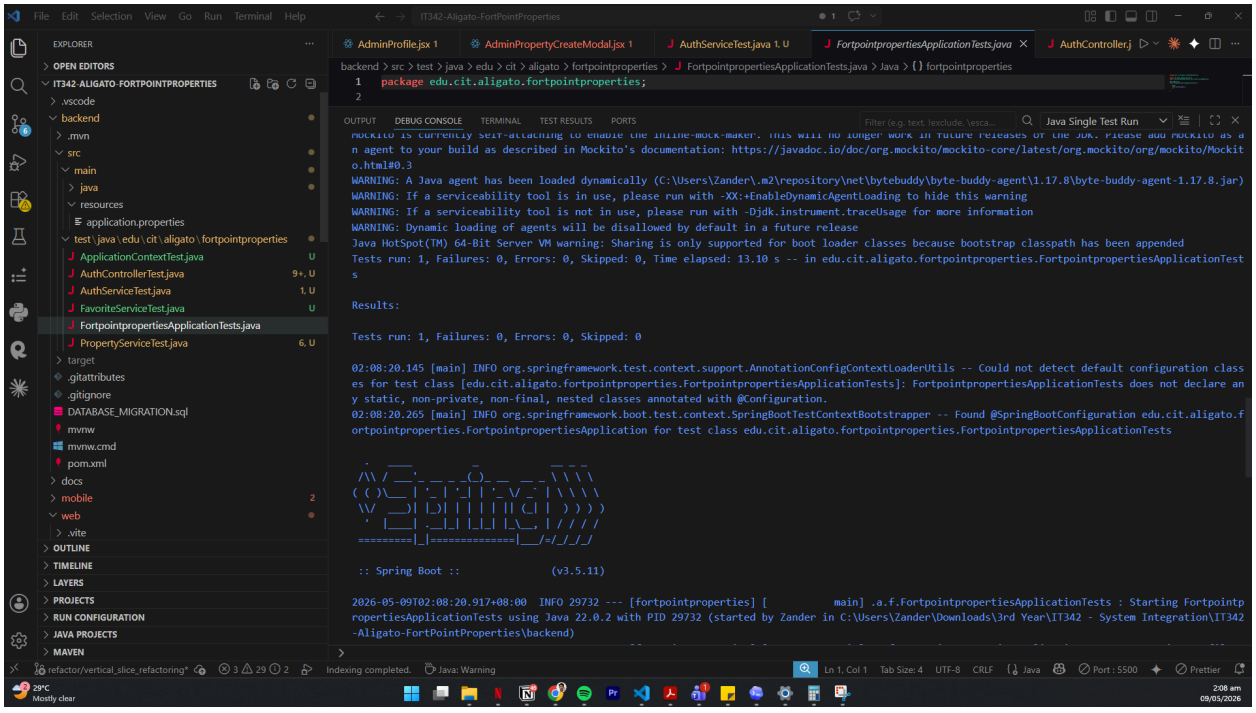


Figure 9: ApplicationContextTest (Build Success)

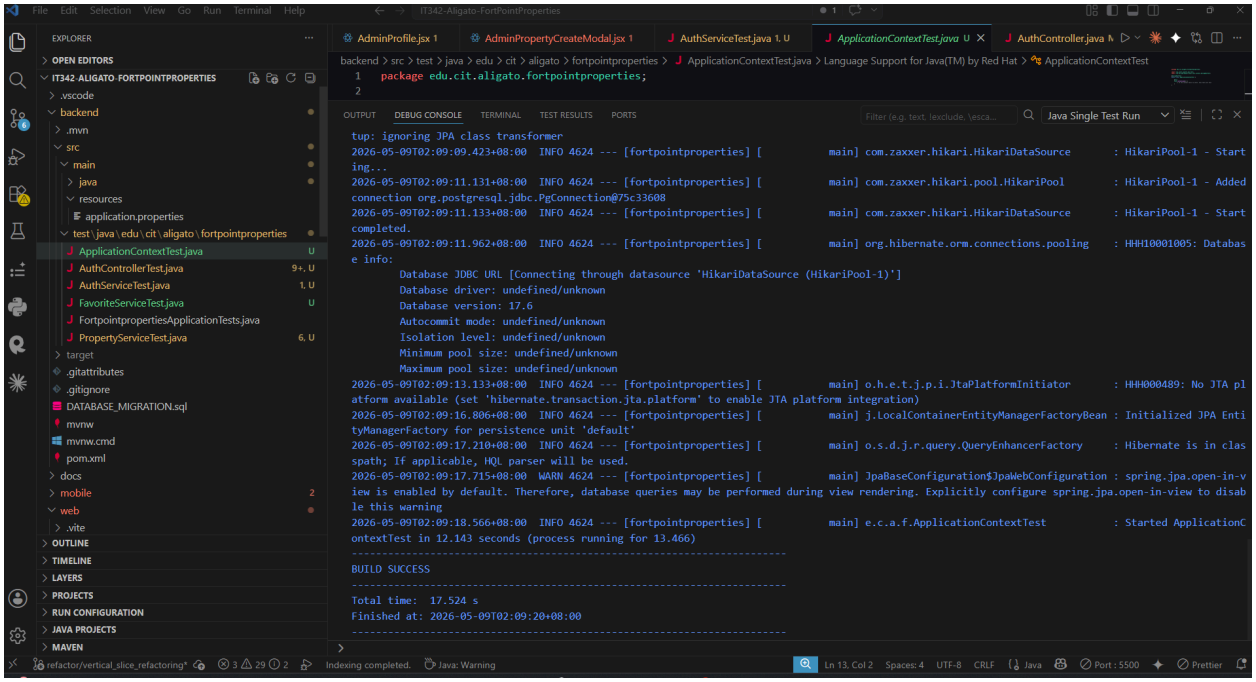
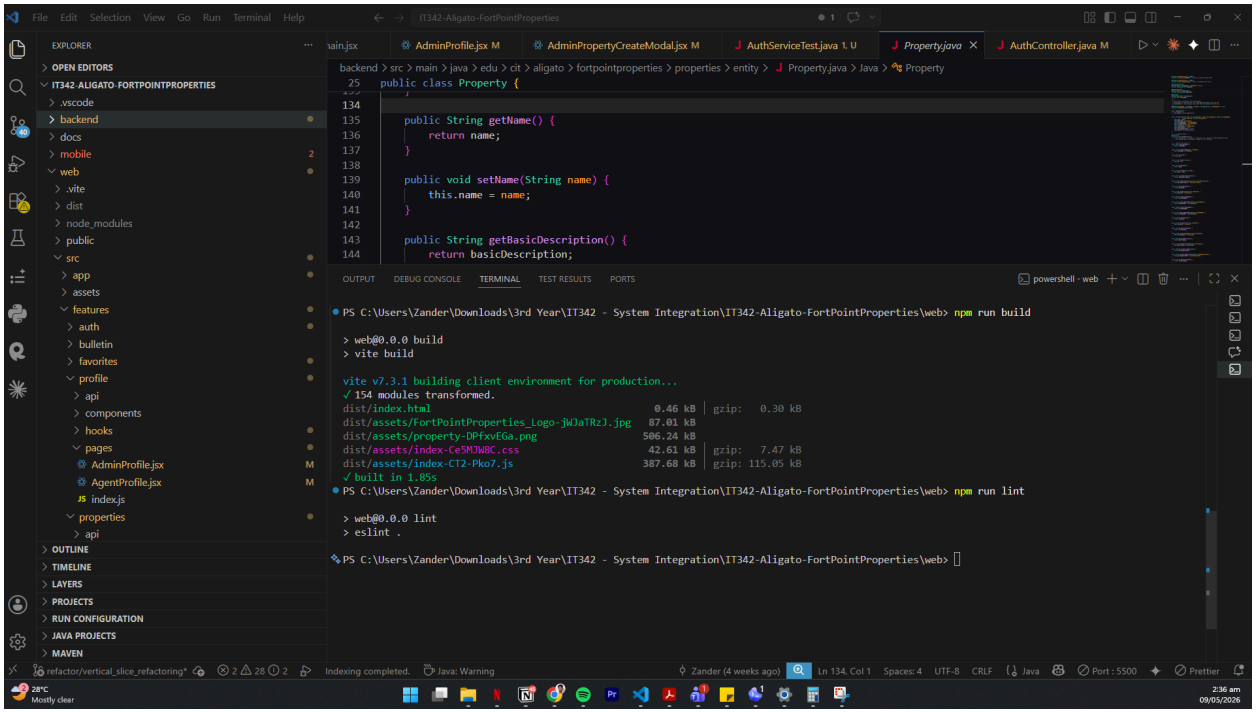


Figure 10: Frontend: npm run lint & npm run build passed successfully



Frontend verification was done using linting and production build validation, while UI behavior was confirmed through manual testing

5.2 Issues Found and Fixes

Issues	Root Cause	Fixes Applied
Backend test coverage was incomplete	Existing tests covered only a few happy paths and did not fully validate negative cases or feature behavior	Added automated backend tests for duplicate email registration, weak password validation, invalid login, profile lookup, property CRUD, property unit CRUD, search/filter behavior, favorites handling, DTO mapping, and count/check methods
Articles feature caused frontend lint errors	Articles files were placeholders with unused variables and no backend implementation yet	Removed the placeholder articles feature folder and removed related article routes/navigation links
Registration form failed lint due to regex escape	Password special-character regex contained an unnecessary escaped bracket	Replaced the regex check with a clearer special-character list validation
Profile pages had React hook dependency warnings	`useEffect` called `fetchProfile` without including it in the dependency array	Added `fetchProfile` to the dependency arrays in admin and agent profile pages
Admin property create modal had an undefined variable	`errors[listingType]` referenced `listingType` as a variable instead of the `listingType` field	Changed the check to `errors.listingType`
Admin property details modal triggered React hook lint error	`setFormData` was called synchronously inside `useEffect` to derive state from props	Refactored form initialization so derived property form data is created during initial state setup
Property search hook had an unused variable	`isLoggedIn` was destructured but not used	Removed the unused `isLoggedIn` variable

Auth context triggered Fast Refresh warning	`AuthContext.jsx` exported both a component and a non-component hook/context utility	Split the context and hook into separate files while keeping `AuthProvider` in `AuthContext.jsx`
Production build failed after context split	Windows path resolution conflicted between `AuthContext.jsx` and `authContext.js` due to similar casing	Renamed the shared context file to `AuthContextBase.js` and updated imports